

04 · Matriz de confusión y métricas: cómo se calculan y cómo se llaman bien

Machine Learning · UTB 2026-II · Mini-solver: pega tus predicciones y sale todo

2026-10-09

Tabla de contenidos

1. Para qué sirve este documento	1
2. La matriz de confusión: el orden importa	1
3. El mini-solver	2
4. Las llamadas correctas (argumentos por nombre)	4
5. Los dos errores y el negocio	4
6. Casos esquina (para no perder puntos por descuido)	4

1. Para qué sirve este documento

Referencia rápida de **todas** las métricas de clasificación binaria: la matriz de confusión (con el orden exacto que devuelve scikit-learn), la llamada correcta de cada métrica con **argumentos por nombre**, y un **mini-solver** al que le pegas tus vectores `y_real` e `y_pred` y te devuelve la tabla completa con interpretación.

2. La matriz de confusión: el orden importa

Con **positivo = mora = 1** (la convención del curso):

	Predicho 1 (mora)	Predicho 0 (paga)
Real 1 (mora)	VP (verdadero positivo)	FN (falso negativo: se coló)
Real 0 (paga)	FP (falso positivo: marqué a un bueno)	VN (verdadero negativo)

En scikit-learn, `confusion_matrix(y_true, y_pred, labels=[0, 1])` devuelve las celdas en ese orden de clases, y `**.ravel()` las aplana así:

VN, FP, FN, VP

— es decir: **fila por fila** (primero la fila del 0, luego la del 1). Ese orden es LA trampa clásica: desempaquetar como VP, FN, FP, VN da todos los valores intercambiados.

```
# =====
# IMPORTS mínimos: solo métricas. Este documento no entrena nada:
# trabaja con vectores de predicciones que tú le pasas.
# =====
import numpy as np
import pandas as pd
from IPython.display import Markdown, display
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix)

def texto(md):
    """Imprime Markdown enriquecido."""
    display(Markdown(md))

# Demostración de una vez por todas del orden de .ravel():
# una matriz inventada con valores distintos en cada celda.
demo = confusion_matrix(y_true=[0, 0, 1, 1], y_pred=[0, 1, 0, 1],
                       labels=[0, 1]) # labels=[0,1] FIJA el orden de clases
print(demo) # [[VN, FP], [FN, VP]]
VN, FP, FN, VP = demo.ravel() # el orden correcto al desempaquetar
print(f"VN={VN} FP={FP} FN={FN} VP={VP}")

[[1 1]
 [1 1]]
VN=1 FP=1 FN=1 VP=1
```

3. El mini-solver

```
# =====
#
# CAMBIA AQUÍ: pega tus vectores de verdad y de predicción.
# 1 = positivo (mora / sí tomó), 0 = negativo (paga / no tomó).
#
# Este ejemplo es la versión V06 de la actividad: exactitud 0.625
# contra tasa base 0.750 -> NO la supera (el caso interesante).
y_real = [0, 1, 0, 0, 1, 0, 0, 0] # <- lo que de verdad pasó
y_pred = [0, 0, 1, 0, 1, 0, 0, 1] # <- lo que predijo tu árbol

def reporte_metricas(y_real, y_pred, nombre_positivo="mora",
                    nombre_negativo="paga"):
    """Calcula matriz, métricas e interpretación desde dos vectores.

    Devuelve un DataFrame con todo. La interpretación de cada métrica
    se escribe en el lenguaje del banco (positivo=negocio riesgoso).
    """
    y_real = np.asarray(y_real); y_pred = np.asarray(y_pred)
    n = len(y_real)
    p = y_real.mean() # proporción de positivos
    # ---- matriz de confusión: labels=[0,1] fija filas/columnas 0 y 1
    cm = confusion_matrix(y_true=y_real, y_pred=y_pred, labels=[0, 1])
    VN, FP, FN, VP = cm.ravel() # ¡VN, FP, FN, VP!
    # ---- métricas a mano (para enseñar) y con sklearn (para verificar)
    exactitud_manual = (VP + VN) / n
    precision_manual = VP / (VP + FP) if VP + FP else 0.0
    recall_manual = VP / (VP + FN) if VP + FN else 0.0
```

```

f1_manual = (2 * precision_manual * recall_manual /
             (precision_manual + recall_manual)
             if precision_manual + recall_manual else 0.0)
tasa_base = max(p, 1 - p) # el tonto que dice
                        # siempre la mayoritaria
# ---- verificación: sklearn debe dar EXACTAMENTE lo mismo
assert np.isclose(exactitud_manual, accuracy_score(y_true=y_real, y_pred=y_pred))
assert np.isclose(precision_manual, precision_score(y_true=y_real,
                                                    y_pred=y_pred, pos_label=1, zero_division=0))
assert np.isclose(recall_manual, recall_score(y_true=y_real,
                                              y_pred=y_pred, pos_label=1))
assert np.isclose(f1_manual, f1_score(y_true=y_real, y_pred=y_pred,
                                      pos_label=1))

filas = [
    ("tasa base", "max(p, 1-p)", tasa_base),
    ("exactitud", "(VP+VN)/n", exactitud_manual),
    ("precisión", "VP/(VP+FP)", precision_manual),
    ("recall", "VP/(VP+FN)", recall_manual),
    ("F1", "2·P·R/(P+R)", f1_manual),
]
tab = pd.DataFrame(filas, columns=["métrica", "fórmula", "valor"]).round(3)
tab["¿qué dice?"] = [
    f"acertar prediciendo siempre '{nombre_positivo if p >= 0.5 else nombre_negativo}'",
    f"de cada {n} clientes, acierta en {VP+VN}",
    f"de los {VP+FP} que mandó a {nombre_positivo}, {VP} eran de verdad",
    f"de los {VP+FN} {nombre_positivo} reales, atrapó {VP}",
    "media armónica de precisión y recall"]
return tab, cm, (VP, FN, FP, VN, tasa_base, exactitud_manual)

tab, cm, (VP, FN, FP, VN, tasa_base, exactitud) = reporte_metricas(y_real, y_pred)

# ---- matriz bonita con totales (como aparece en los informes):
cm_df = pd.DataFrame(cm,
                     index=[f"real 1 (mora)", "real 0 (paga)"],
                     columns=[f"pred. 1 (mora)", f"pred. 0 (paga)"])
cm_df["total"] = cm_df.sum(axis=1)
cm_df.loc["total"] = cm_df.sum()
display(cm_df)
display(tab)

```

	pred. 1 (mora)	pred. 0 (paga)	total
real 1 (mora)	4	2	6
real 0 (paga)	1	1	2
total	5	3	8

	métrica	fórmula	valor	¿qué dice?
0	tasa base	$\max(p, 1-p)$	0.750	acertar prediciendo siempre 'paga'
1	exactitud	$(VP+VN)/n$	0.625	de cada 8 clientes, acierta en 5
2	precisión	$VP/(VP+FP)$	0.333	de los 3 que mandó a mora, 1 eran de verdad
3	recall	$VP/(VP+FN)$	0.500	de los 2 mora reales, atrapó 1
4	F1	$2 \cdot P \cdot R / (P+R)$	0.400	media armónica de precisión y recall

```
# La lectura que el parcial pide: ¿supera la tasa base? y el consejo de
# precisión/recall según el costo del error.
supera = exactitud > tasa_base
print(f"Exactitud {exactitud:.3f} vs tasa base {tasa_base:.3f} ->",
      "Sí la supera: el modelo aprendió algo." if supera else
      "NO la supera: peor que predecir la mayoritaria a ciegas; no sirve.")
print(f"VP={VP} FN={FN} FP={FP} VN={VN} | (las 4 celdas suman {VP+FN+FP+VN})")
```

Exactitud 0.625 vs tasa base 0.750 -> NO la supera: peor que predecir la mayoritaria a ciegas; no sirve.
VP=1 FN=1 FP=2 VN=4 | (las 4 celdas suman 8)

4. Las llamadas correctas (argumentos por nombre)

```
# CÓMO SE LLAMAN BIEN las métricas: con y_true / y_pred explícitos y,
# en las que lo tienen, con pos_label=1 (positivo = clase 1).
# Estas líneas son 'la respuesta' al punto 15 de la actividad.
clase = y_pred # tus predicciones 0/1
exactitud = accuracy_score(y_true=y_real, y_pred=clase)
precision = precision_score(y_true=y_real, y_pred=clase,
                           pos_label=1, zero_division=0) # zero_division:
                                                         # qué devolver si
                                                         # VP+FP=0 (evita el
                                                         # warning/errores)

recall = recall_score(y_true=y_real, y_pred=clase, pos_label=1)
f1 = f1_score(y_true=y_real, y_pred=clase, pos_label=1)
VN2, FP2, FN2, VP2 = confusion_matrix(y_true=y_real, y_pred=clase,
                                       labels=[0, 1]).ravel()
print(dict(exactitud=round(exactitud,3), precision=round(precision,3),
          recall=round(recall,3), f1=round(f1,3),
          VN=VN2, FP=FP2, FN=FN2, VP=VP2))
```

```
{'exactitud': 0.625, 'precision': 0.333, 'recall': 0.5, 'f1': 0.4, 'VN': np.int64(4), 'FP': np.int64(2), 'FN': np.int64(1), 'VP': np.int64(1)}
```

5. Los dos errores y el negocio

error	qué pasó	costo para el banco
FN	era moroso, lo di por buen pagador	el caro: prestó y no le pagan (pierde capital)
FP	pagaba, lo mandé a mora	perder el negocio (perdió la ganancia, no el capital)

- Si el **FN** es el error caro → querer **más RECALL** (atrapar morosos), aunque caiga la precisión.
- Si el **FP** fuera el caro (p. ej. pierdo clientes valiosos) → querer más **precisión**.

Precisión y recall tiran en direcciones opuestas: subir el umbral de “riesgoso” sube la precisión (solo marco a los muy claros) y baja el recall (se me escapan más). El **F1** es el compromiso: media armónica que castiga desequilibrios (si una vale 0, F1 vale 0).

6. Casos esquina (para no perder puntos por descuido)

- **VP + FP = 0** (nadie predicho positivo): precisión indefinida → `zero_division=0` la devuelve como 0 sin warning.
- **VP + FN = 0** (no hay positivos reales): recall indefinido → ídem.

- **Hoja empatada** (mitad y mitad): scikit-learn **predice la clase 0** (lo dice la actividad: “si una hoja queda empatada, predice 0”).
- **No fijar `labels=[0,1]`**: si el modelo solo predijo una clase, la matriz viene incompleta (2x2 recortada) y `.ravel()` da otro orden — **siempre** pasar `labels=[0, 1]`.
- **Exactitud con clases desbalanceadas** puede engañar: con 95 % de buenos pagadores, decir “todos buenos” ya acierta 95 % — por eso se compara contra la **tasa base** y se miran precisión/recall.